

IDEATOR GECT

Centre for Innovation · Government Engineering College Thrissur

STRUCTURAL INSPECTION DRONE

Software Team — Execution Plan

Month 1 · Crack Detection Module

Version	v2.0 · March 2026
Classification	Internal · Software Team

1. Project Overview

The Structural Inspection Drone (SID) is an autonomous inspection system built by IDEATOR GECT to automate structural health monitoring of civil infrastructure — primarily bridges. SID integrates RGB, IR, and stereo cameras with AI-driven defect detection, mapped onto a 3D BIM model.

This document is the execution plan for the software team covering Month 1 — the Crack Detection Module. It covers member responsibilities, weekly tasks, the repository structure, training details, the JSON output schema, and handoff rules.

2. Team Structure

The team has 8 members. Where possible, members from the same batch are grouped together for easier coordination.

- S2 CSE — Devika + Viswajith M P (Group B · Crack Analysis Layer — paired)
- S6 ECE — Aswin + Hridhya (Group A · Evaluation, Group C · Pipeline)
- S6 CSE — Sreedha + Maria (Group C · API, Group D · Corrosion Dataset — paired)
- S4 ECE — Sreehari (Group D · Dataset support)
- S4 CSE — Lead (Integration, QA, schema)

Name	Batch	Role	Responsibility
Gabriel	S4 CSE	Software Lead	Integration, QA, JSON schema, coordination
Devika	S2 CSE	Analysis — Geometry	Pixel geometry extraction (paired with Viswajith)
Viswajith M P	S2 CSE	Analysis — Severity	Severity logic + JSON output (paired with Devika)
Aswin	S6 ECE	Model Evaluation	mAP / precision / recall, retraining decision
Hridhya	S6 ECE	Real-Time Pipeline	Live video inference, FPS optimisation
Sreedha	S6 CSE	API Development	Flask/FastAPI endpoint + session logging
Maria	S6 CSE	Corrosion Dataset	Dataset collection + CVAT annotation
Sreehari	S4 ECE	Corrosion Dataset	Image sourcing and quality filtering

3. GitHub Repository

All code, weights, and configuration files live in the shared repository. Clone it and work within the structure described below. Do not share code files over WhatsApp — commit everything to GitHub.

REPO

<https://github.com/gabsqj/sid-software-team-ideator>

3.1 Repository Structure

Files already in the repository are marked. Placeholders show where each member's work should be committed.

SID-Software/	
├── SID_Software_Plan_Month1.docx	← this document
├── crack/	
│ ├── crack.ipynb	← MAIN TRAINING FILE (run on Google Colab)
│ ├── crack_dataset.yaml	← dataset config (classes, train/val
├── paths)	
│ ├── convert_to_yolo.py	← converts raw dataset to YOLO format
│ ├── Colab_Training_Guide.md	← step-by-step guide to reproduce training
│ ├── documentation.md	← project notes and references
│ ├── yolol1n-seg.pt	← base nano model (kept for reference
├── only)	
│ ├── results.png	training uses yolol1s-seg.pt (small)
│ ├── best.pt	← training metrics graph (all 90 epochs)
│ ├── crack_dataset/	← TRAINED WEIGHTS — use for all inference
│ │ ├── images/	20.5 MB · YOLO11s-seg · 90 epochs
│ │ └── labels/	
│ ├── analysis/	← raw dataset folder
│ │ ├── geometry.py	← Devika + Viswajith (Week 3)
│ │ ├── severity.py	
│ │ └── output_schema.json	
│ ├── pipeline/	← Hridhya (Week 4)
│ │ └── realtime.py	
│ ├── api/	← Sreedha (Week 4)
│ │ └── app.py	
├── corrosion_dataset/	← Sreehari + Maria (Week 3-4)
│ ├── images/	
│ └── annotations/	

3.2 The Dataset ZIP File

The YOLO-formatted dataset (crack_yolo_data.zip) is not in the repository — too large. It was generated from the raw dataset using convert_to_yolo.py and is shared via Google Drive.

ZIP

<https://drive.google.com/file/d/1tmmljYEAbh6Ng9Bt0vUibe6p62VnvyAv/view?usp=sharing>
— crack_yolo_data.zip (YOLO format, generated by convert_to_yolo.py)

- Download this file before running crack.ipynb on Colab.
- Place it in your Google Drive at MyDrive/crack_yolo_data.zip — the notebook expects this exact path.
- Do not re-upload or duplicate it on Drive; use the shared link.

3.3 The best.pt File

best.pt is the trained YOLOv11s-seg weight file saved at the epoch with the highest validation mAP across all 90 training epochs. It is the single source of truth for all inference work.

PATH	crack/best.pt — 20.5 MB · YOLO11s-seg fused · 114 layers · 10M parameters
-------------	---

- All inference code must load best.pt — not last.pt.
- Do not overwrite best.pt unless Aswin confirms a retrained model has better mAP on unseen images. In that case, rename the new file best_v2.pt.
- Model loading: `model = YOLO('crack/best.pt')`

4. How the Model Was Trained

Training was done on Google Colab using the free GPU runtime (Tesla T4). The full training code is in crack/crack.ipynb. Below is a summary of each cell so anyone can reproduce or continue training if needed.

FILE	crack/crack.ipynb — the only file needed to reproduce training
-------------	--

4.1 Notebook — Cell by Cell

```
# Cell 1 — Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Cell 2 — Copy dataset ZIP from Drive, extract, install ultralytics
!cp "/content/drive/MyDrive/crack_yolo_data.zip" /content/
!unzip -q /content/crack_yolo_data.zip -d /content/crack_data/
!pip install ultralytics

# Cell 3 — Fix dataset YAML paths for the Colab environment
# Updates crack_dataset.yaml so paths point to /content/crack_data/

# Cell 4 — Load model and train (90 epochs, 640px, batch 16)
from ultralytics import YOLO
model = YOLO('yolov11s-seg.pt') # small segmentation — switched from nano
results = model.train(
    data='/content/crack_data/crack_dataset.yaml',
    epochs=100,
```

```
imgsz=640,
batch=16,
device=0,
project='/content/drive/MyDrive/YOLO_Crack_Training',
name='results'
)

# Cell 5 - Verify: run inference on a test image
model_path = '/content/drive/MyDrive/YOLO_Crack_Training/results/weights/best.pt'
model = YOLO(model_path)
results = model.predict(source='test_image.jpg', save=True, conf=0.25)

# Cell 6 - Save best.pt and results.png back to Drive
!cp ../weights/best.pt "/content/drive/MyDrive/YOLO_Crack_Training/results/"
!cp ../results.png "/content/drive/MyDrive/YOLO_Crack_Training/results/"
```

4.2 Reproducing Training

- Open crack/crack.ipynb in Google Colab (File → Open → GitHub or upload directly).
- Confirm crack_yolo_data.zip is in your Drive at MyDrive/crack_yolo_data.zip.
- Run all cells in order. Weights save automatically to Google Drive under YOLO_Crack_Training/results/.
- After training, download best.pt from Drive and replace crack/best.pt in the repo. Do the same for results.png.
- Note: albumentations augmentation (Blur, MedianBlur, CLAHE, ToGray) kicks in automatically from epoch 81 onwards — this is expected behaviour.

5. Training Results — Final Epoch Metrics

The model completed 90 epochs in 5.07 hours on a Tesla T4. The metrics below are from the final validation run on best.pt (the epoch with highest mAP), confirmed by the post-training validation pass.

MODEL	YOLO11s-seg · fused · 114 layers · 10,067,203 parameters · 32.8 GFLOPs · 7.0ms inference
-------	--

Metric	Value	
Epochs completed	90 / 90 (5.07 hours total)	
GPU memory used	5.94 GB (Tesla T4)	
Input resolution	640 × 640	
Training images	1,444	
Total instances	2,614	
Model size	20.5 MB (best.pt, optimizer stripped)	
Parameters	10,067,203 (114 layers, 32.8 GFLOPs)	
Inference speed	7.0 ms per image (0.2ms pre + 3.1ms post)	

Metric	Value	
Box Precision	0.862 (86.2%)	✓
Box Recall	0.633 (63.3%)	
Box mAP@50	0.716 (71.6%)	✓
Box mAP@50-95	0.529 (52.9%)	
Mask Precision	0.779 (77.9%)	
Mask Recall	0.549 (54.9%)	
Mask mAP@50	0.573 (57.3%)	
Mask mAP@50-95	0.120 (12.0%)	
box_loss (final epoch)	0.8492	
seg_loss (final epoch)	1.627	
cls_loss (final epoch)	0.7109	
dfl_loss (final epoch)	1.100	

5.1 Training Screenshot

Paste the final epoch terminal screenshot (showing Epoch 90/90, GPU_mem, box_loss, mAP etc.) here for the team's reference:

```
Epoch  GPU_mem  box_loss  seg_loss  cls_loss  dfl_loss  sem_loss  Instances  Size
90/90   5.94G    0.8492    1.627     0.7109    1.1        0         30         640: 100%
Class   Images   Instances  Box(P)    R         mAP50    mAP50-95)  Mask(P)    R         mAP50    mAP50-95): 100%
all     1444     2614      0.863     0.633     0.716    0.529      0.778     0.549     0.573     0.12      46/46 2.0it/s 22.5s
```

5.2 Training Metrics Graph (results.png)

The results.png file (auto-generated by YOLO) shows loss curves, mAP, precision, and recall across all 90 epochs. It is committed to crack/results.png in the repository. Paste a copy here:

available in crack/results.png in the repository

5.3 Evaluation Summary

The trained model already meets all evaluation targets. Aswin should verify these numbers hold on unseen real-world images (not from the training/validation set) before signing off.

Metric	Current result	Minimum	Status
Box mAP@50 (unseen images)	0.716 ✓	0.70	Passed — model is ready
Box Precision	0.862 ✓	0.80	Passed
Box Recall	0.633	0.60	Passed
Confidence threshold	0.45 (default)	0.45	Adjust based on FP/FN ratio

6. Month 1 — Week-by-Week Plan

Month 1 delivers the complete Crack Detection Module. Corrosion dataset collection runs in parallel from Week 3 onwards.

Week 1 — Completed

Done: Pre-annotated crack dataset acquired. YOLO11s-seg trained on Google Colab (crack.ipynb) for 90 epochs — 5.07 hours total. best.pt (20.5 MB) and results.png committed to the repository. Final validation mAP@50: 0.716, Box Precision: 0.862.

Week 2 — Model Evaluation

Owner: Aswin (S6 ECE). Target: wrap this up by Day 3 so Hridhya can start pipeline work without waiting.

Week 2	Member	Tasks
<i>Evaluation</i>	Aswin (S6 ECE)	- Clone the repo and confirm best.pt is at crack/best.pt (20.5 MB) - Run inference on the held-out validation set and record mAP50, precision, recall — compare against Section 5 - Collect 20–30 real-world crack images not from the training set (bridge surfaces, concrete walls) and test - Log false positives (non-cracks detected) and false negatives (cracks missed) - If mAP50 on real-world images is below 0.65, flag it in the group — a retraining decision will follow - If performance is acceptable, confirm weights in the group chat and Hridhya can proceed - Write eval_report.md in crack/ with test images, results, and verdict
	Lead — parallel	- Commit output_schema.json to crack/analysis/ (full schema in Section 7 of this document) - Brief Devika and Viswajith on their Week 3 tasks

Week 2	Member	Tasks
	Sreehari (S4 ECE) — parallel	- Start sourcing corrosion images from Kaggle, Roboflow, Google Images - Target: 500+ images across rust patches, exposed rebar, surface flaking

Week 3 — Crack Analysis Layer

Owners: Devika and Viswajith M P (both S2 CSE). They work as a direct pair — Devika's geometry.py output feeds straight into Viswajith's severity.py. Sync daily.

NOTE	mm-based crack width is not possible yet — no stereo camera. All severity uses relative_width (pixel domain). The geometry_mm fields in JSON must stay null with stereo_available: false until Month 4.
-------------	---

Week 3	Member	Tasks
<i>Analysis</i>	Devika (S2 CSE)	- Load best.pt: model = YOLO('crack/best.pt') and run on 50 test images - From each bbox [x1,y1,x2,y2]: length_px = bbox diagonal, width_px = min(bbox_w, bbox_h) - Compute relative_width = width_px / image_width_px - Write geometry.py: extract_geometry(bbox, img_w, img_h) -> dict - Verify relative_width values are between 0 and 1 on all 50 images - Commit geometry.py to crack/analysis/ on GitHub
	Viswajith M P (S2 CSE)	- Take Devika's geometry output (relative_width) as input - Implement: Minor < 0.01 · Moderate 0.01–0.03 · Severe > 0.03 - Visually check 20 detections — if labels look wrong, recalibrate thresholds - Implement generate_output(detection, geometry, severity, session_meta) -> JSON matching output_schema.json exactly - Batch test on 50 images and export to results_batch_test.json - Commit severity.py to crack/analysis/ on GitHub
	Sreehari + Maria — parallel	- Continue corrosion collection — target 300+ images this week - Sort into folders: rust_patches/ · exposed_rebar/ · surface_flaking/ - Remove blurred, very low-res (<200px), or irrelevant images

Week 4 — Real-Time Pipeline and API

Owners: Hridhya (S6 ECE) for the live pipeline and Sreedha (S6 CSE) for the API. Both depend on best.pt being confirmed by Aswin.

Week 4	Member	Tasks
Pipeline + API	Hridhya (S6 ECE)	- Load best.pt and run inference frame-by-frame on a USB camera or video file - For each frame, call geometry.py then severity.py from Week 3 - Target > 15 FPS throughput (reduce input to 416 if needed) - Overlay bounding boxes with severity labels on the live stream using OpenCV - Pipe each frame's JSON detection to a queue for the API - Commit realtime.py to crack/pipeline/ on GitHub
	Sreedha (S6 CSE)	- Build Flask or FastAPI app in crack/api/app.py - POST /detect — accepts image, returns JSON detection matching output_schema.json - GET /session/{session_id} — returns all detections for a session - Log each detection to logs/{session_id}.json and logs/{session_id}.csv - Test with Postman: upload a crack image, confirm JSON matches the schema - Test with a real photo from a mobile camera (cracked wall / concrete surface) - Commit app.py and a short test_results.md to crack/api/
	Lead — integration + demo	- Integrate all four components: model → geometry → severity → API - Run the full end-to-end demo: live camera → inference → JSON → API → logged file - Record a short screen capture for the team review - Review all commits and give written feedback to each member
	Maria (S6 CSE)	- Begin CVAT annotation on corrosion images collected by Sreehari and herself - Classes: corrosion · steel_exposed · rust - Target: 200 annotated images by end of Week 4 - Export in YOLO format and commit to corrosion_dataset/annotations/

7. Official JSON Output Schema

This is the schema all detection output must conform to. Viswajith generates it in severity.py. Sreedha's API returns this format. Hridhya's pipeline passes detections through it. The file is committed to crack/analysis/output_schema.json.

FILE

crack/analysis/output_schema.json — committed to repository

7.1 Full Schema

```
{
  /* — Session metadata — */
  "session_id": "sid_20250316_143022",
  "timestamp": "2025-03-16T14:30:22.481Z",
  "source": "usb_camera",
  "frame_id": 142,
  "image_file": "frame_142.jpg",
  "image_width_px": 1280,
  "image_height_px": 720,

  /* — Location — */
  "location": {
    "latitude": null,
    "longitude": null,
    "altitude_m": null,
    "label": "bridge_deck_faceA"
  },

  /* — Model — */
  "model": {
    "name": "yololls-seg",
    "weights": "best.pt",
    "input_resolution": "640x640"
  },

  /* — Detections — */
  "detections": [
    {
      "detection_id": "det_142_001",
      "bbox_px": [100, 120, 250, 160],
      "confidence": 0.94,

      "geometry_px": {
        "length_px": 158,
        "width_px": 40,
        "relative_width": 0.031
      },

      "geometry_mm": {
        "width_mm": null,
        "depth_z_mm": null,
        "stereo_available": false
      },

      "severity": {
        "level": "Moderate",
        "basis": "relative_width",
        "thresholds_used": {
          "minor_below": 0.01,
          "moderate_below": 0.03
        }
      }
    },
  ],
}
```

```

    "is456_compliance": {
        "permissible_width_mm": null,
        "exceeds_limit": null,
        "exposure_class": null
    }
  },
  /* — Summary — */
  "summary": {
    "total_detections": 1,
    "severity_counts": { "Minor": 0, "Moderate": 1, "Severe": 0 },
    "highest_severity": "Moderate",
    "flagged": false
  }
}

```

7.2 Field Reference

Required = always present. Now = populated in Month 1. Month 4/5 = intentionally null until hardware is integrated.

Field	Type	Status	Notes
session_id	string	Required	Format: sid_YYYYMMDD_HHMMSS — unique per run
timestamp	string	Required	ISO 8601 UTC — datetime.utcnow().isoformat()
source	string	Required	usb_camera video_file zed2 rgb_camera
frame_id	integer	Required	Monotonically increasing per session
location.label	string	Now	Manual zone label — e.g. bridge_deck_faceA
location.latitude/longitude	float null	Month 5	Populated when GPS (pynmea2) integrated
bbox_px	int[4]	Required	[x1,y1,x2,y2] top-left→bottom-right from YOLO
confidence	float	Required	0.0–1.0 · filter below 0.45
geometry_px.relative_width	float	Now	width_px / image_width_px — severity basis now
geometry_mm.width_mm	float null	Month 4	Requires ZED2: Width = (B × f) / d
geometry_mm.stereo_available	bool	Required	Always false until ZED2 integrated
severity.level	string	Required	Minor Moderate Severe
severity.basis	string	Required	Log what severity was computed from
is456_compliance.*	object	Month 4	IS 456:2000 permissible crack widths
summary.flagged	bool	Required	true if any Severe detection in this frame

7.3 Severity Logic

Viswajith implements this in severity.py. Starting thresholds below — calibrate against real detections before the batch test.

```
def classify_severity(relative_width: float) -> str:
```

```
MINOR_THRESHOLD    = 0.01    # calibrate against real images
MODERATE_THRESHOLD = 0.03    # calibrate against real images

if relative_width < MINOR_THRESHOLD:
    return 'Minor'
elif relative_width < MODERATE_THRESHOLD:
    return 'Moderate'
else:
    return 'Severe'

# mm-based severity deferred to Month 4 (ZED2 stereo integration)
# IS 456:2000 permissible limits:
# Mild:      0.30 mm   Moderate: 0.20 mm   Severe: 0.10 mm
```

8. Environment Setup

Use the same Python environment to avoid version conflicts.

8.1 Installation

```
git clone https://github.com/gabsgj/sid-software-team-ideator
cd sid-software-team-ideator

python -m venv venv
source venv/bin/activate      # Mac / Linux
venv\Scripts\activate        # Windows

pip install -r requirements.txt

# requirements.txt:
# ultralytics>=8.0.0
# opencv-python>=4.8
# flask>=3.0    (or fastapi>=0.110 + uvicorn + python-multipart)
# numpy>=1.24
```

8.2 Loading the Model

```
from ultralytics import YOLO

model = YOLO('crack/best.pt')    # always use best.pt

results = model.predict('test_image.jpg', conf=0.45, imgsz=640)

for result in results:
    for box in result.boxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
        confidence = float(box.conf[0])
        print(f'Crack at [{x1},{y1},{x2},{y2}]   conf={confidence:.2f}')
```

9. GitHub and Communication Rules

9.1 GitHub Rules

- All code must be committed to GitHub — no sharing .py files over WhatsApp.
- Commit messages should be clear: feat: add geometry.py crack width extractor
- Create a branch named yourname/task (e.g. devika/geometry) and open a pull request when done.
- Do not push directly to main.
- Do not modify another member's files without discussing it first.

9.2 Critical Dependencies

Aswin must confirm best.pt on real-world images by Day 3 of Week 2. Hridhya cannot begin pipeline work until weights are signed off. If evaluation is taking longer, raise it in the group and a call will be made on whether to proceed.

output_schema.json must be committed before Viswajith starts severity.py and before Sreedha starts the API. Both depend on it. This will be done at the start of Week 2.

Devika and Viswajith are a direct pair — Devika's geometry.py output is the input to Viswajith's severity.py. Sync daily and do not work in isolation.

10. End of Month 1 — Deliverables

Everything below should be complete and in the repository by end of Week 4.

Deliverable	Owner	Status
Trained YOLOv11s-seg model — best.pt	Lead	Done ✓
Evaluation report on unseen images	Aswin	Week 2
geometry.py — pixel crack dimensions	Devika	Week 3
severity.py — classification + JSON	Viswajith M P	Week 3
realtime.py — live inference pipeline	Hridhya	Week 4
app.py — Flask/FastAPI + logging	Sreedha	Week 4
Corrosion images — 300+ organised	Sreehari	Week 3–4
Corrosion annotations — 200+ YOLO format	Maria	Week 4
End-to-end demo recording	Lead	Week 4